

第九章：网络协议栈

引言

本章导读

本章是一次在教学操作系统领域的原创性探索。我们首次在 uCore 教学操作系统中实现了完整的 TCP/IP 网络协议栈，并成功从内核态直接调用外部 AI API，为后续 AI 小镇项目奠定了网络通信基础。

本章的创新贡献：

- 提出了"用户态协议栈"到"内核态协议栈"的移植框架**：基于 microps 教学协议栈，设计了一套完整的平台适配层，解决了内存管理、互斥锁、调度原语等跨平台问题。
- 解决了 VirtIO Legacy 模式与 Modern 模式的兼容性问题**：QEMU 虚拟网卡默认使用 Legacy 模式，而现有代码仅支持 Modern 模式，我们完成了完整的 Legacy 模式驱动实现。
- 设计了"轮询+中断"混合等待机制**：在没有完整进程调度的内核中实现阻塞 I/O，解决了中断处理与忙等待的死锁问题。
- 实现了从操作系统内核直接访问互联网的能力**：成功调用 OpenAI API，验证了网络协议栈的完整性和正确性。

在现代计算机系统中，网络通信已经成为不可或缺的基础设施。然而，大多数教学操作系统（如 xv6、uCore）都没有实现网络功能，学生对"操作系统如何处理网络数据包"缺乏直观认识。本章将填补这一空白。

为什么需要网络协议栈？

在前面的章节中，我们实现的操作系统只能与本地资源交互：从键盘读取输入、向屏幕输出文字、读写磁盘文件。但现实中的计算机系统几乎都需要联网：

- 访问网页需要 HTTP 协议
- 发送邮件需要 SMTP 协议
- 远程登录需要 SSH 协议
- 云服务调用需要 REST API

这些高层协议都建立在 TCP/IP 协议栈之上。没有网络协议栈，操作系统就是一座"信息孤岛"。

注意

- 本章尚未实现https，使用http，其中，119.3.217.132:8000为测试服务器

本章目标

本章将实现一个完整的 TCP/IP 网络协议栈，根据代码结构，包括：

协议层	实现内容
应用层	HTTP 客户端（用于调用 AI API）
传输层	TCP（可靠传输）、UDP（不可靠传输）

协议层	实现内容
网络层	IP（路由）、ICMP（ping）
链路层	ARP（地址解析）、以太网帧处理
驱动层	VirtIO 网络设备驱动

实践体验

获取本章代码：

源码已上传大赛官网

在 qemu 模拟器上运行本章代码：

```
$ make BASE=1 CHAPTER=9
$ make run
>> ch9_test
virtio-net: initialized successfully
virtio-net: IP configured: 10.0.2.15/24, gateway: 10.0.2.2
ch9: Testing AI API...
ch9: AI response: Hello! How can I assist you today?
```

看到 AI 的回复，说明网络协议栈工作正常——我们的操作系统已经能够访问互联网了！

本章代码树

```
os-btbu/net/
├─ platform.h      # uCore平台适配层头文件
├─ platform.c      # 平台适配实现（时间、随机数、字符串函数）
├─ util.h          # 工具函数头文件
├─ util.c          # 工具函数（队列、字节序转换、校验和）
├─ net.h           # 网络设备抽象层头文件
├─ net.c           # 网络核心层实现
├─ ether.h         # 以太网层头文件
├─ ether.c         # 以太网帧处理
├─ ip.h            # IP层头文件
├─ ip.c            # IP协议实现（路由表、分片）
├─ arp.h           # ARP层头文件
├─ arp.c           # 地址解析协议实现
├─ icmp.h          # ICMP层头文件
├─ icmp.c          # ICMP协议实现（ping响应）
├─ udp.h           # UDP层头文件
├─ udp.c           # UDP协议实现
├─ tcp.h           # TCP层头文件
├─ tcp.c           # TCP协议实现（完整状态机）
├─ http.c          # HTTP客户端实现
└─ virtio_net.c    # VirtIO网络设备驱动
```

协议栈架构

分层模型

网络协议栈采用经典的分层架构，每层只与相邻层交互：



数据流向

发送数据（自顶向下）：

1. 应用层构造 HTTP 请求
2. TCP 层分段、添加序号和校验和
3. IP 层添加源/目的地址
4. ARP 解析下一跳 MAC 地址
5. 以太网层封装成帧
6. VirtIO 驱动发送到网卡

接收数据（自底向上）：

1. 网卡中断通知数据到达
2. 以太网层解析帧头，判断协议类型
3. IP 层验证目的地址，查找路由
4. TCP 层重组数据、确认收到
5. 应用层读取完整响应

核心数据结构

网络设备 (net_device)

网络设备是对物理或虚拟网卡的抽象：

```
struct net_device {
    struct net_device *next;      /* 设备链表 */
    struct net_iface *ifaces;    /* 网络接口链表 */
    unsigned int index;          /* 设备索引 */
    char name[IFNAMSIZ];         /* 设备名如 "net0" */
    uint16 type;                 /* 设备类型 */
    uint16 mtu;                 /* 最大传输单元 */
    uint16 flags;               /* 设备标志 */
    uint8 addr[NET_DEVICE_ADDR_LEN]; /* MAC地址 */
    struct net_device_ops *ops; /* 设备操作函数 */
    void *priv;                 /* 私有数据 */
};
```

IP 接口 (ip_iface)

IP 接口记录网络地址配置：

```
struct ip_iface {
    struct net_iface iface;
    struct ip_iface *next;
    ip_addr_t unicast;          /* IP地址 */
    ip_addr_t netmask;          /* 子网掩码 */
    ip_addr_t broadcast;        /* 广播地址 */
};
```

TCP 控制块 (tcp_pcb)

TCP 控制块维护每个连接的状态：

```
struct tcp_pcb {
    int state;                  /* TCP状态机状态 */
    struct ip_endpoint local;    /* 本地端点 */
    struct ip_endpoint foreign; /* 远程端点 */
    struct {
        uint32 nxt, una;        /* 发送序号 */
        uint16 wnd;             /* 发送窗口 */
    } snd;
    struct {
        uint32 nxt;             /* 接收序号 */
        uint16 wnd;             /* 接收窗口 */
    } rcv;
    uint8 buf[65535];          /* 接收缓冲区 */
    struct sched_ctx ctx;       /* 调度上下文 */
    struct queue_head queue;    /* 重传队列 */
};
```

```
};
```

VirtIO 网络驱动

理解 VirtIO

VirtIO 是一种半虚拟化技术，虚拟机中的操作系统"知道"自己运行在虚拟环境中，通过标准化接口与宿主机通信，性能接近原生硬件。

QEMU 模拟的 VirtIO 网卡位于 MMIO 地址 0x10002000，支持：

- 两个 Virtqueue：RXQ（接收）和 TXQ（发送）
- 每个队列 8 个描述符

Legacy 模式 vs Modern 模式

这是我们在调试中遇到的第一个重大问题。VirtIO 有两个版本：

特性	Legacy模式 (v1)	Modern模式 (v2)
队列内存	连续两页对齐内存	分散的desc/avail/used
队列地址	QUEUE_PFN (页帧号)	QUEUE_DESC_LOW/HIGH等
页大小	需要设置GUEST_PAGE_SIZE	不需要
FEATURES_OK	不需要	必须设置

QEMU 默认使用 Legacy 模式，但很多参考代码只实现了 Modern 模式。我们必须按 Legacy 模式初始化：

```
/* ch9: Legacy模式队列初始化 */
static void virtq_init(struct virtq *vq, int sel, int num) {
    *R(VIRTIO_MMIO_QUEUE_SEL) = sel;
    *R(VIRTIO_MMIO_QUEUE_NUM) = num;
    memset(vq->pages, 0, sizeof(vq->pages));

    /* ch9: 使用页帧号而非绝对地址 */
    *R(VIRTIO_MMIO_QUEUE_PFN) = ((uint64)vq->pages) >> PGSHIFT;

    /* ch9: 队列布局: desc在页首, avail紧随其后, used在第二页 */
    vq->desc = (struct virtq_desc *)vq->pages;
    vq->avail = (struct virtq_avail *) (vq->pages + NUM * sizeof(struct virtq_desc));
    vq->used = (struct virtq_used *) (vq->pages + PGSIZE);
}
```

数据收发流程

发送流程：

1. 分配 TX 描述符
2. 填充 virtio-net 头部

3. 复制数据到发送缓冲区
4. 添加到可用环 (Available Ring)
5. 通知设备 (写 QUEUE_NOTIFY 寄存器)

接收流程:

1. 中断触发 (设备写入已用环)
2. 从已用环 (Used Ring) 获取描述符
3. 解析 virtio-net 头部
4. 将数据传递给以太网层
5. 回收描述符到可用环

平台适配层

设计思想

microps 协议栈原本运行在用户态，依赖标准 C 库。移植到 uCore 内核需要重新实现这些依赖：

```
/* ch9: 内存管理适配 */
void *memory_alloc(uint size) {
    /* 使用 kalloc 分配整页，简单但有效 */
    return kalloc();
}

void memory_free(void *ptr) {
    kfree(ptr);
}

/* ch9: 互斥锁适配 */
typedef struct spinlock mutex_t;
#define mutex_lock(m)    acquire(m)
#define mutex_unlock(m) release(m)

/* ch9: 调度适配 - 关键! */
int sched_sleep(struct sched_ctx *ctx, mutex_t *mutex, ...) {
    /* 必须在等待时开启中断，否则网卡中断无法到达 */
    mutex_unlock(mutex);

    while (!ctx->ready) {
        intr_on(); /* ch9: 开中断! */

        /* ch9: 主动轮询网络事件 */
        if (net_pending & INTR_IRQ_SOFTIRQ) {
            net_softirq_handler();
        }
        net_timer_handler();
    }

    mutex_lock(mutex);
    return 0;
}
```

"轮询+中断"混合机制

这是我们设计的一个关键机制。在没有完整进程调度的内核中实现阻塞 I/O 面临一个矛盾：

- 需要等待网络数据到达
- 网络数据到达时会触发中断
- 但如果关闭中断等待，中断永远不会到达

解决方案：在等待循环中保持中断开启，同时主动轮询网络事件。

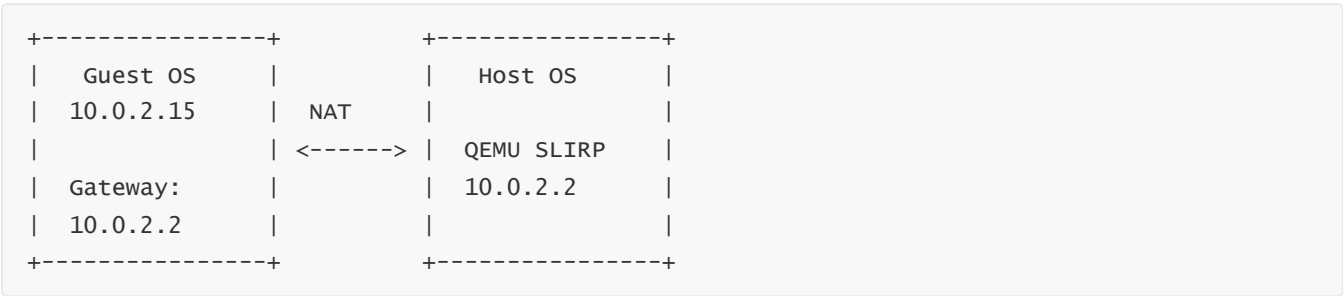
QEMU 网络配置

启动参数

```
qemu-system-riscv64 \
-machine virt \
-device virtio-net-device,netdev=net0 \
-netdev user,id=net0,hostfwd=tcp::8080-:80
```

网络拓扑（user 模式）

QEMU user 模式使用 NAT，Guest OS 位于私有网络：



默认配置：

- Guest IP: 10.0.2.15
- 网关: 10.0.2.2
- DNS: 10.0.2.3
- 子网掩码: 255.255.255.0

调试记录

这是一次在未知领域的全新探索。以下记录了调试过程中遇到的关键问题。

Bug 1: IP路由失败

现象： `[ERROR]ip_route_get_iface() failure`

原因： `virtio_net_init()` 只初始化了驱动，没有配置 IP 地址和路由。

解决：在驱动初始化后添加 IP 配置：

```
iface = ip_iface_alloc("10.0.2.15", "255.255.255.0");
ip_iface_register(dev, iface);
ip_route_set_default_gateway(iface, "10.0.2.2");
```

Bug 2: 字节序问题

现象：IP 地址解析后路由仍然失败。

原因：`parse_ip()` 和 `hton32()` 双重转换。

解决：统一使用网络字节序存储 IP 地址。

Bug 3: VirtIO Legacy 模式

现象：`virtio-net: device not found` (version=1)

原因：代码只支持 Modern 模式 (version=2) 。

解决：重写队列初始化，使用 `QUEUE_PFN` 寄存器。

Bug 4: 等待时死锁

现象：程序卡在 `sched_sleep()` 不返回。

原因：关中断等待，导致网卡中断无法到达。

解决：在等待循环中开启中断，主动轮询网络事件。

API 参考

网络初始化

```
int net_init(void);           /* 初始化协议栈 */
int net_run(void);           /* 启动网络 */
void virtio_net_init(void);   /* 初始化VirtIO网卡 */
```

TCP 接口

```
int tcp_open(void);           /* 创建TCP套接字 */
int tcp_connect(int id, struct ip_endpoint *ep); /* 连接服务器 */
int tcp_send(int id, uint8 *data, uint len);    /* 发送数据 */
int tcp_receive(int id, uint8 *buf, uint size); /* 接收数据 */
int tcp_close(int id);        /* 关闭连接 */
```

HTTP 客户端

```
/* ch9: 调用AI API示例 */
int ai_chat(const char *prompt, char *response, int max_len);
```


文件修改清单

文件	内容
net/*.c	完整的网络协议栈实现
os/main.c	添加网络初始化调用
os/trap.c	添加网卡中断处理
os/plic.c	添加 VIRTIO1_IRQ 处理
Makefile	添加 net 目录编译规则

知识点总结

- 1. **网络分层模型**：每层只关注自己的职责，通过标准接口与相邻层交互
- 2. **VirtIO 半虚拟化**：虚拟机与宿主机协作，提高 I/O 性能
- 3. **字节序问题**：网络字节序（大端）vs 主机字节序（小端）
- 4. **中断与轮询**：阻塞 I/O 需要正确处理中断时机

思考题

- 1. 如果不使用 QEMU user 模式，而是使用 tap 模式，网络配置需要做哪些改变？
- 2. 当前 TCP 实现不支持拥塞控制，在高延迟网络下会有什么问题？
- 3. 如何实现一个简单的 DNS 客户端，让系统支持域名解析？

下一步：ch10 进化调度

在实现了网络通信能力之后，下一章我们将开始构建 AI 小镇的基础设施——进化调度系统，让 NPC 进程在"自然选择"的压力下竞争生存。